OPENCUES · ACTUATOR REFERENCE

Volume & brightness: states & gestures

These two blanks aren't fills that drop a dead number. They're live knobs. This is every state they move through and every key that moves them. Shipped behaviour, verified live.

Download PDF ↓ Download video ↓

```
volume 26%
↳ system volume   (ctrl+alt+up/down to adjust)
```

volume

Labelled `system volume`. Each press moves it 6 percentage points, between 0% and 100%. Backed by `volume-blank.sh get|set` (Windows / macOS / PipeWire / PulseAudio / ALSA).

brightness

Labelled `screen brightness`. Each press moves it 10 percentage points, between 0% and 100%. Same script shape, same gestures, so everything here applies to both.

01 Entry: how you get into a value

Include a number and it **sets** the level; leave it out and it just **reads** the current one. Either way you end up with the value in your text and the cursor on it.

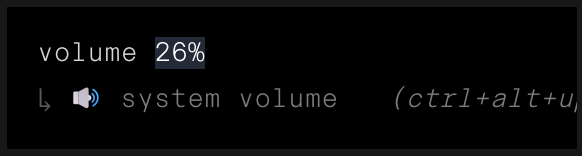
YOU TYPE	MODE	YOUR TEXT BECOMES	WHAT HAPPENS
`volume _`	GET	`volume 26%`	reads the current level and writes it in
`volume is _`	GET	`volume 26%`	a few words in between still work
`volume 40 _`	SET	`volume 40%`	turns the volume to 40%, writes it in
`set volume to 60 _`	SET	`volume 60%`	turns the volume to 60%

The word `volume` stays in your text so the number still makes sense on its own later (`volume 40%`, not a stray `40%`).

02 The note: what you see on the value

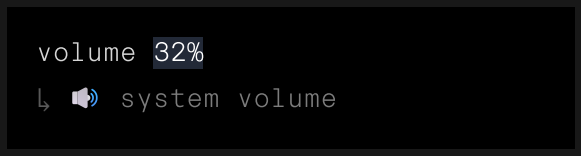
Standard `emoji · label · (hint)` format. The label persists (the standing "this is a live knob" signal); the one-time hint teaches the gesture, then drops after your first adjust.

FIRST ENCOUNTER



Label + the teaching hint.

AFTER YOUR FIRST ADJUST



Hint gone, label stays: still visibly live.

03 Gestures: what moves the value

Put the cursor on the number. Every press changes the real device and updates the number, never going below 0% or above 100%.

KEY	DOES	AT THE EDGES
`Ctrl+Alt+↑`	raises it one step (6% for volume, 10% for brightness)	stops at 100%
`Ctrl+Alt+↓`	lowers it by the same step	stops at 0% (muted / screen dark)
`_`	`NEW` nudges it up one step, the quick one-key climb	climbs to 100% and stays there; the `_` is never typed into your text

`_` is the one-direction quick nudge (up); use `Ctrl+Alt+↓` when you need to come back down. Both write straight to the device.

04 The value states

One continuous 0–100 range with two special edges. The number in the number in your text is the real value; the bars below are just to picture it.

NORMAL: ANYWHERE BETWEEN

32% moves up or down a step at a time

76%

TOP EDGE: 100%

100% the maximum: pressing up again does nothing (it never wraps round to 0%)

```
volume 100%
↳ 🔊 system volume
```

BOTTOM EDGE: 0%

0% silent / screen dark: pressing down again does nothing

```
volume 0%
↳ 🔊 system volume
```

There is no "keep as is" and no countdown, because a dial has neither. It has a current value and directions. The clamp is felt, not labelled: the number simply stops moving at 0 and 100.

05 Worked example: climbing with `_`

Verified live. `volume _` reads the current level (26%), then each bare `_` adds 6 points until it reaches 100% and stops.

`volume _`

reads 26%

```
volume 26%
↳ 🔊 system volume    (ctrl+alt+up/down to adjust)
```

`_`

26 + 6 = 32% · louder

```
volume 32%
↳ 🔊 system volume
```

`_`

32 + 6 = 38%

```
volume 38%
↳ 🔊 system volume
```

...keep pressing `_` ...

at 98%, +6 would be 104%, so it stops at 100%

```
volume 100%
↳ 🔊 system volume
```

`_` again at the top

nothing changes, and no `_` is typed

```
volume 100%
↳ 🔊 system volume
```

`Ctrl+Alt+↓`

100 – 6 = 94% · quieter

```
volume 94%
↳ 🔊 system volume
```

> **It stops at the top, it does not wrap.** Holding `_` down at 100% keeps it at 100%; it never rolls round to 0%. That is deliberate: one extra press should not silently mute you. To come back down, use `Ctrl+Alt+↓`.

Reflects shipped behaviour: `defaults/blanks/{volume,brightness}/BLANK.md` + `dim-render.ts` (note) + `cycling.ts` (`cycleBlankStep`, caret-armed `step()`, `_`-nudge in `stepUnderscore`). Verified live on OpenCode + Claude Code. Values write straight to the device via the blank's script; a host with no audio/brightness backend is a silent no-op.